

Cây khung nhỏ nhất & Đường đi ngắn nhất

Trần Vĩnh Đức - Nguyễn Lâm Tùng

Ngày 1 tháng 12 năm 2023

Cây khung nhỏ nhất & Đường đi ngắn nhất

Trần Vĩnh Đức - Nguyễn Lâm Tùng

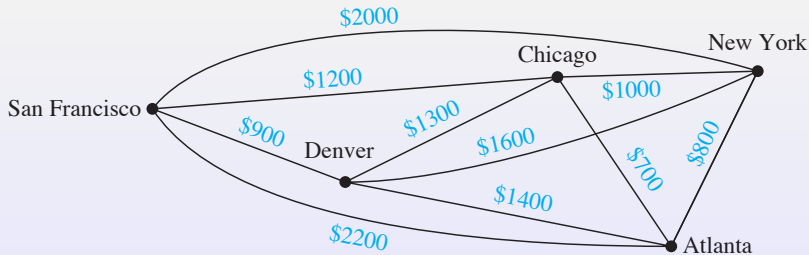
Ngày 1 tháng 12 năm 2023

Cây khung nhỏ nhất

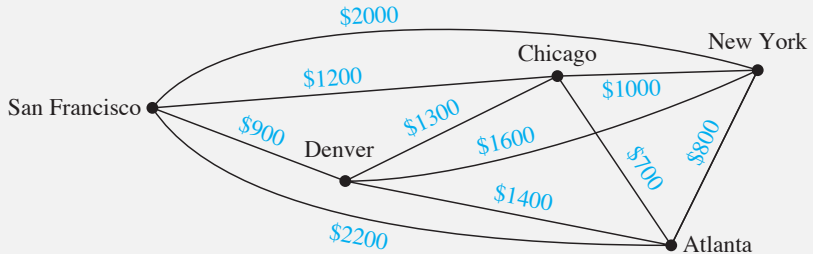
Định nghĩa

Cây khung nhỏ nhất trong một đồ thị liên thông có trọng số là một cây khung có tổng trọng số trên các cạnh của nó là nhỏ nhất.

Hãy tìm cây khung nhỏ nhất của đồ thị sau



Ví dụ



Chọn	Cạnh	Chi phí
1	{Chicago, Atlanta}	700\$
2	{Atlantan, New York}	800\$
3	{Chicago, San Francisco}	1200\$
4	{San Francisco, Denver}	800\$
Tổng chi phí:		3600\$

Thuật toán Prim

Ý tưởng của thuật toán:

- Bắt đầu bằng việc chọn 1 cạnh có trọng số nhỏ nhất để đặt vào cây khung.
- Lần lượt ghép vào cây khung cạnh có trọng số bé nhất trong tất cả các cạnh liên thuộc với cây khung đã có và khi ghép vào không tạo thành chu trình.
- Thuật toán dừng khi $n - 1$ cạnh được ghép vào cây (nếu đồ thị ban đầu có n đỉnh).

Chứng minh tính đúng đắn của thuật toán Prim

- Giả sử lần lượt các cạnh được chọn theo thuật toán Prim là $e_1, e_2, \dots, e_{n-1}$ và S là cây gồm các cạnh trên, S_k là cây với các cạnh là $e_1, e_2, \dots, e_k$. Ký hiệu $\delta(G)$ là tổng trọng số của đồ thị G .
- Gọi k là số nguyên lớn nhất sao cho tồn tại cây khung nhỏ nhất T chứa các cạnh $e_1, e_2, \dots, e_k$ nhưng không tồn tại cây khung nhỏ nhất nào chứa các cạnh $e_1, e_2, \dots, e_k, e_{k+1}$. Thuật toán Prim đúng nếu chứng minh được $S = T$.
- Giả sử $S \neq T$ khi đó $k < n - 1$, do T không chứa e_{k+1} nên khi thêm e_{k+1} vào T sẽ tạo thành chu trình đơn chứa e_{k+1} , hơn nữa chu trình này phải chứa ít nhất 1 cạnh không thuộc S_{k+1} vì S_{k+1} là cây.
- Xuất phát từ 1 đỉnh là đầu mút chung của e_{k+1} với 1 trong các đỉnh $e_1, e_2, \dots, e_k$, đi dọc theo chu trình trên (chiều mà sẽ qua e_{k+1} cuối cùng) cho tới khi gặp tới khi gặp cạnh e đầu tiên không thuộc S_{k+1} , khi đó nó có 1 đầu mút thuộc S_k .

Chứng minh tính đúng đắn của thuật toán Prim(tiếp)

- Xóa e khỏi T và thêm vào e_{k+1} , ta được đồ thị T' có $n - 1$ cạnh và không có chu trình, do đó T' là cây.
- Do e và e_{k+1} cùng kề với một trong các cạnh $e_1, e_2, \dots, e_k$ nhưng e_{k+1} được chọn ở bước $k + 1$ còn e thì không nên trọng số của e_{k+1} không vượt quá trọng số của e , suy ra $\delta(T') \leq \delta(T)$.
- Mặt khác, do T là cây khung bé nhất nên $\delta(T) \leq \delta(T')$ suy ra $\delta(T') = \delta(T)$.
- Vậy T' cũng là cây khung nhỏ nhất của G mà T' chứa $e_1, e_2, \dots, e_k, e_{k+1}$, điều này mâu thuẫn với giả thiết ban đầu.

Thuật toán Prim

function Prim(Đồ thị G liên thông có trọng số với n đỉnh)

T = cạnh có trọng số nhỏ nhất

For $i = 1$ to $n - 2$ do

begin

e = cạnh có trọng số nhỏ nhất trong G thỏa mãn:

e liên thuộc với một đỉnh thuộc T và

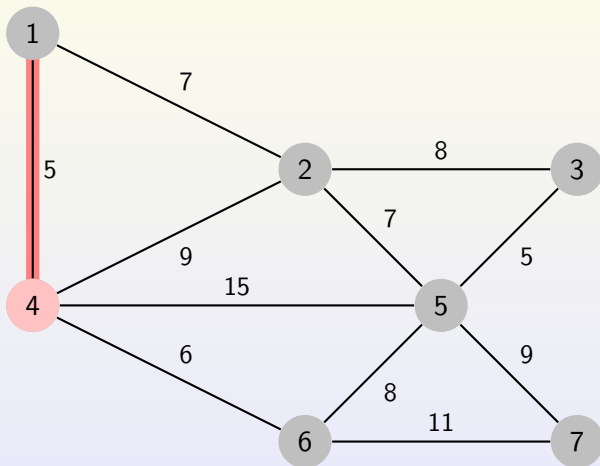
$T \cup \{e\}$ không có chu trình

$T = T \cup \{e\}$

end

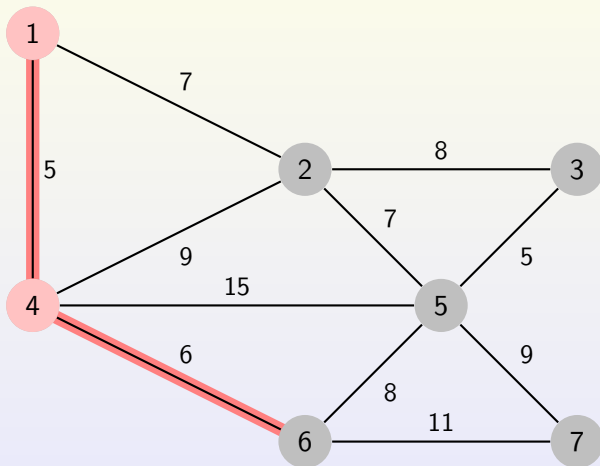
Return T % T là cây khung nhỏ nhất.

Ví dụ: Thuật toán Prim



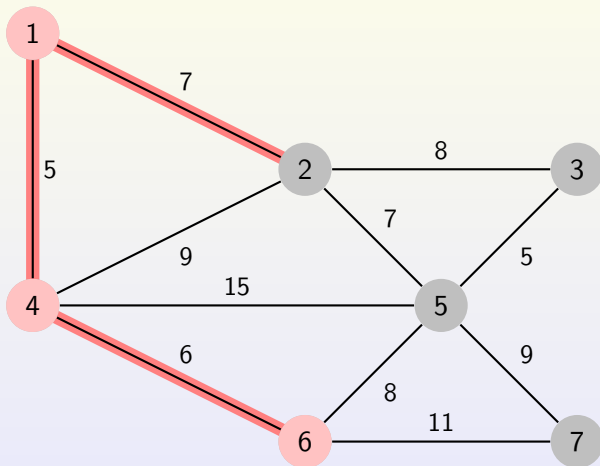
Hình: Nguồn: tikz examples

Ví dụ: Thuật toán Prim



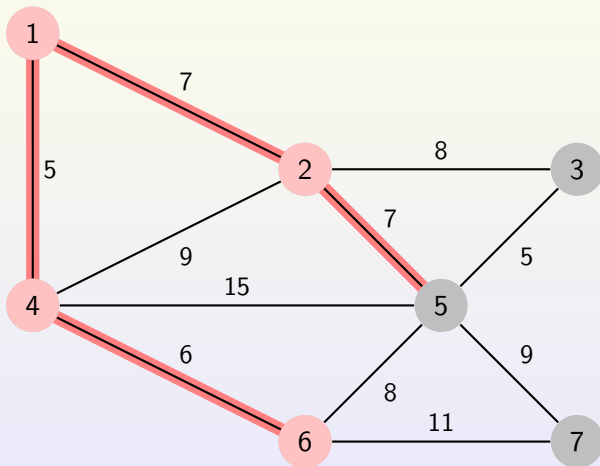
Hình: Nguồn: tikz examples

Ví dụ: Thuật toán Prim



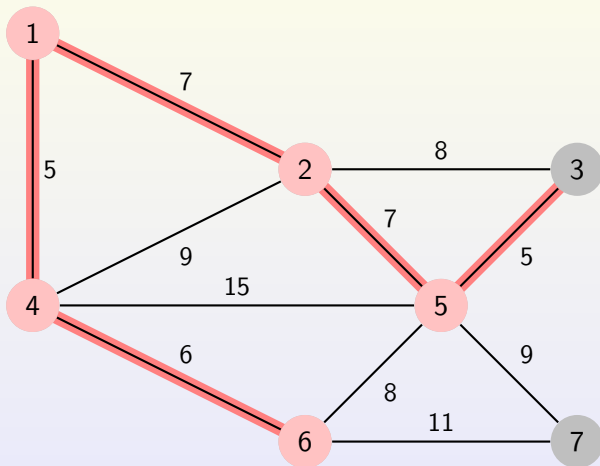
Hình: Nguồn: tikz examples

Ví dụ: Thuật toán Prim



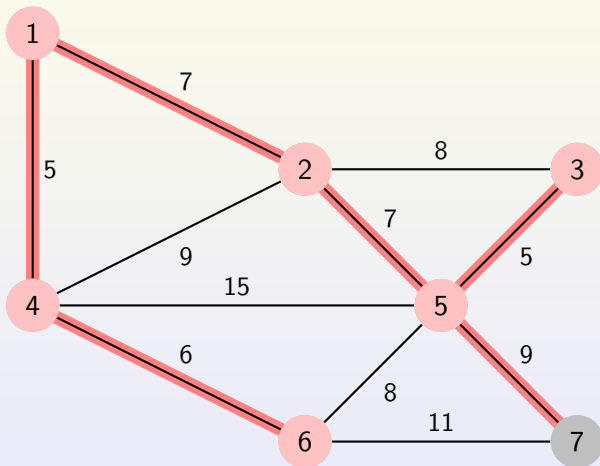
Hình: Nguồn: tikz examples

Ví dụ: Thuật toán Prim



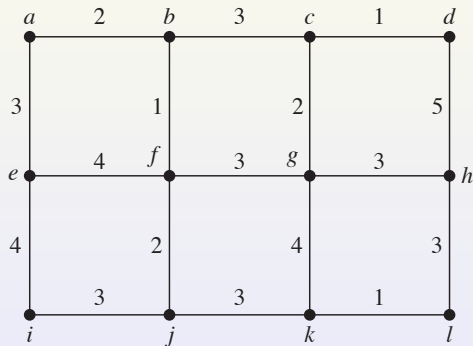
Hình: Nguồn: tikz examples

Ví dụ: Thuật toán Prim



Hình: Nguồn: tikz examples

Ví dụ: Thuật toán Prim



Bước	Cạnh	Trọng số
1	$\{b, f\}$	1
2	$\{a, b\}$	2
3	$\{f, j\}$	2
4	$\{a, e\}$	3
5	$\{i, j\}$	3
6	$\{f, g\}$	3
7	$\{c, g\}$	2
8	$\{c, d\}$	1
9	$\{g, h\}$	3
10	$\{h, l\}$	3
11	$\{k, l\}$	1
Tổng:		24

Thuật toán Kruskal

Ý tưởng của thuật toán:

- Bắt đầu bằng việc chọn 1 cạnh có trọng số nhỏ nhất để đặt vào cây khung T .
- Lần lượt ghép vào cây khung T cạnh có trọng số bé nhất trong các cạnh còn lại của G và khi ghép vào T không tạo thành chu trình.
- Thuật toán dừng khi không thể ghép thêm bất cứ cạnh nào vào T (khi đó T có $n - 1$ cạnh nếu đồ thị G có n đỉnh).

Chứng minh tính đúng đắn của thuật toán Kruskal

- Trước hết ta chứng minh T có n đỉnh. Thật vậy, nếu T có ít hơn n đỉnh thì tồn tại $a \notin T$, nhưng G liên thông nên tồn tại đường đi từ đỉnh v của T đến a là $(v, v_1, v_2, \dots, v_k, a)$.
- Thêm cạnh (v_k, a) vào T sẽ không tạo ra chu trình vì a là đỉnh treo trong T mới, điều này trái với cách xây dựng T . Vậy T có n đỉnh.
- Nếu T không liên thông thì tồn tại 2 đỉnh a, b sao cho không có đường đi từ a đến b (trong T), gọi G_a là thành phần liên thông chứa a , do G liên thông nên tồn tại đường đi trong G từ a đến b , gọi $e = (a_k, s)$ là cạnh đầu tiên trong đường đi đó mà không thuộc G_a .
- Thêm cạnh e vào T sẽ không tạo chu trình (vì nếu có chu trình thì có đường đi khác từ a_k đến s trong G_a , tức là $s \in G_a$), mâu thuẫn với cách xây dựng T .
- Vậy T liên thông và có n đỉnh nên T là cây khung của G .
- Việc chứng minh T là cây khung nhỏ nhất tương tự như trong CM ở thuật toán Prim.

Thuật toán Kruskal

Function Kruskal (Đồ thị G liên thông có trọng số với n đỉnh)

$T =$ đồ thị rỗng

For $i = 1$ to $n - 1$ do

begin

$e =$ cạnh trong G với trọng số nhỏ nhất thỏa mãn

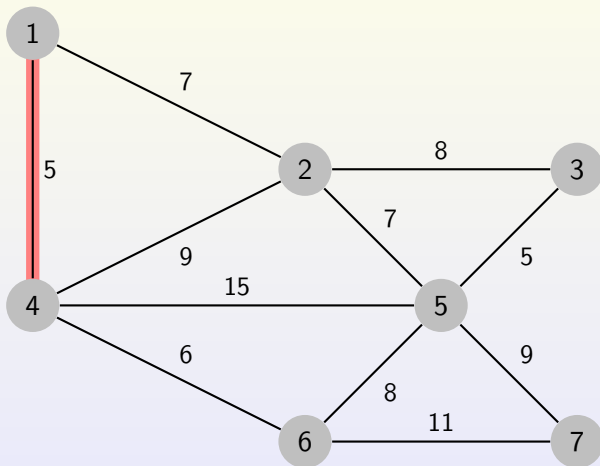
đồ thị $T \cup \{e\}$ không có chu trình

$T = T \cup \{e\}$

end

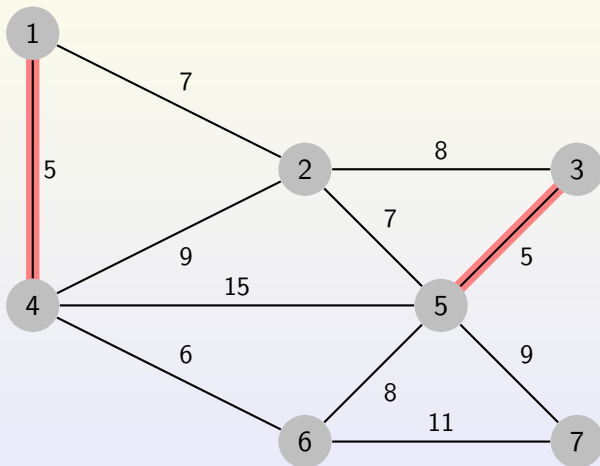
Return T %(T là cây khung nhỏ nhất)

Ví dụ: Thuật toán Kruskal



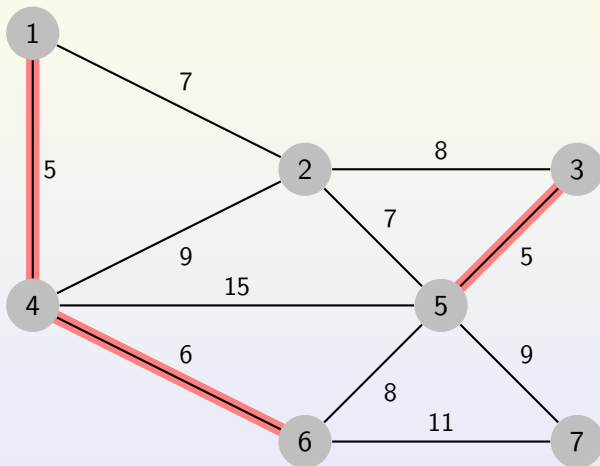
Hình: Nguồn: tikz examples

Ví dụ: Thuật toán Kruskal



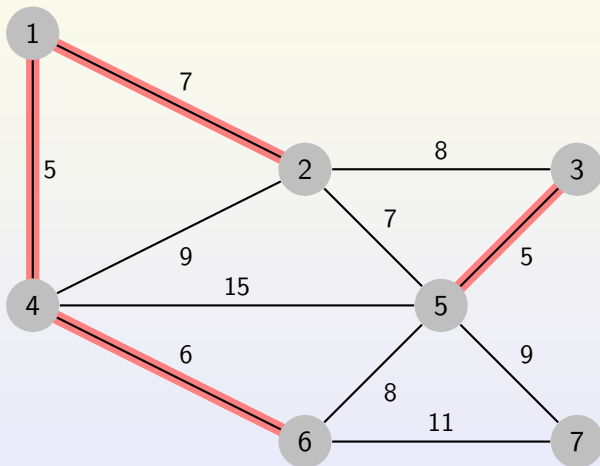
Hình: Nguồn: tikz examples

Ví dụ: Thuật toán Kruskal



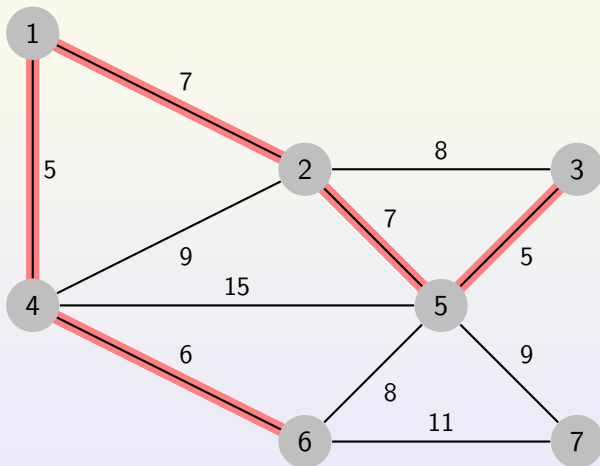
Hình: Nguồn: tikz examples

Ví dụ: Thuật toán Kruskal



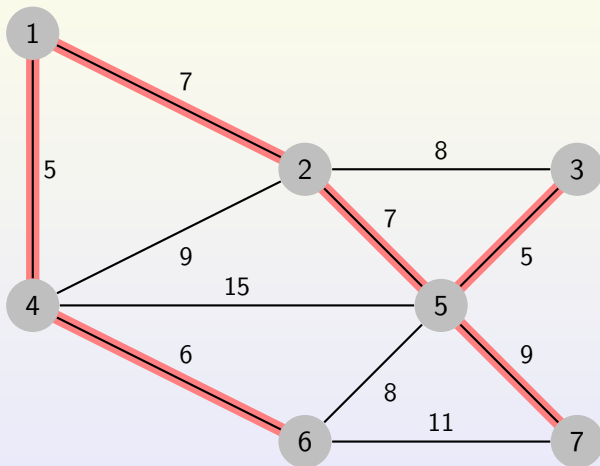
Hình: Nguồn: tikz examples

Ví dụ: Thuật toán Kruskal



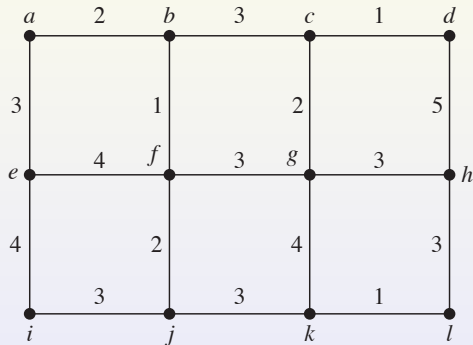
Hình: Nguồn: tikz examples

Ví dụ: Thuật toán Kruskal



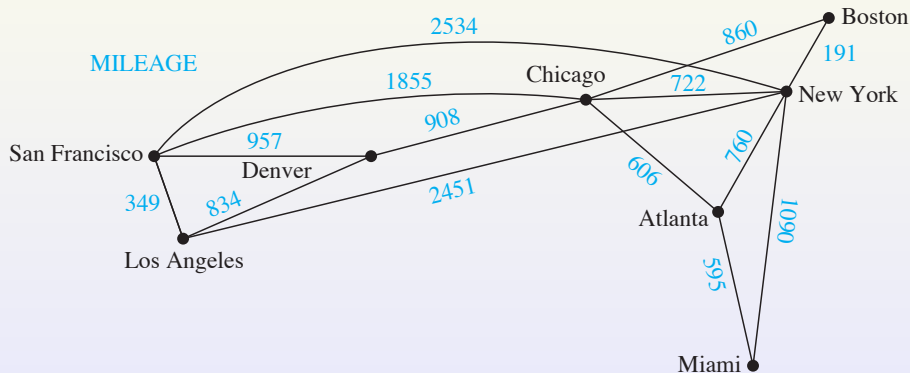
Hình: Nguồn: tikz examples

Ví dụ: Thuật toán Kruskal

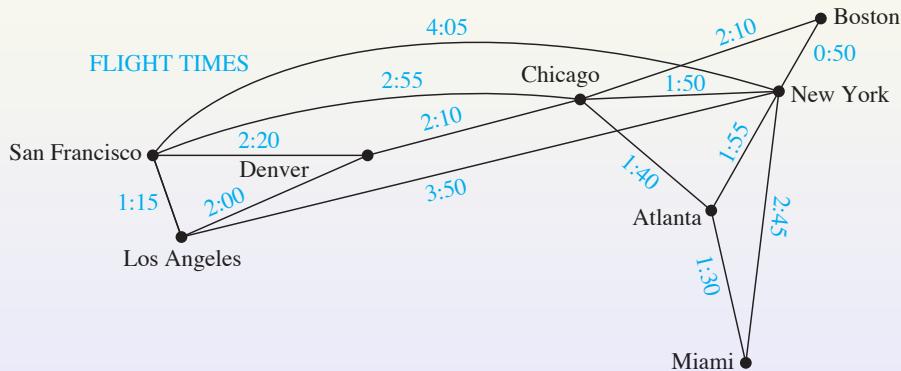


Bước	Cạnh	Trọng số
1	{c, d}	1
2	{k, l}	2
3	{b, f}	2
4	{c, g}	3
5	{a, b}	3
6	{f, j}	3
7	{b, c}	2
8	{j, k}	1
9	{g, h}	3
10	{i, j}	3
11	{a, e}	1
Tổng:		24

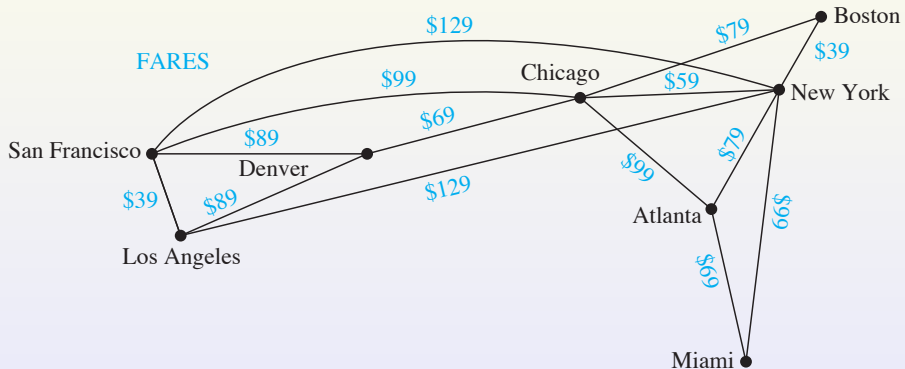
Mô hình đường bay với trọng số là khoảng cách



Mô hình đường bay với trọng số là thời gian bay



Mô hình hệ thống đường bay với trọng số là giá vé



Đồ thị có trọng số

- Đồ thị với mỗi cạnh được gán một trọng số.
- Ký hiệu (V, E, w) trong đó (V, E) là đồ thị và

$$w : E \longrightarrow \mathbb{R}^+$$

là một hàm từ tập cạnh E lên tập số dương (trọng số).

- Độ dài của đường đi

$$p : v_0 \rightarrow v_1 \rightarrow v_2 \rightarrow \cdots \rightarrow v_k$$

là tổng

$$w(p) = w(v_0 \rightarrow v_1) + w(v_1 \rightarrow v_2) + \cdots + w(v_{k-1} \rightarrow v_k).$$

Thuật toán Dijkstra

Bài toán: Tìm đường đi ngắn nhất từ đỉnh a đến đỉnh z trong đồ thị có trọng số $G = (V, E, w)$.

Ý tưởng chung: Ở vòng lặp thứ k , xây dựng tập S_k và dán nhãn $L(v)$ cho các đỉnh $v \in V$ thỏa mãn các điều kiện sau, gọi là điều kiện Dijkstra:

- (1) Nếu $u \in S_k$ thì $L(u)$ là độ dài đường đi ngắn nhất từ a đến u .
- (2) Nếu $u \notin S_k$ và $L(u) < \infty$ thì $L(u)$ là độ dài ngắn nhất trong các đường đi từ a đến u có dạng (gọi là đường đi từ S_k đến u):

$$a \rightarrow v_1 \rightarrow v_2 \rightarrow \cdots \rightarrow v_p \rightarrow u,$$

trong đó $a, v_1, v_2, \dots, v_p \in S_k$

Nhận xét:

- Khi $z \in S_k$ thì ta có $L(z)$ là độ dài đường đi ngắn nhất từ a đến z .
- Khi $S_k = V$ thì $\forall v \in V$, $L(v)$ là độ dài đường đi ngắn nhất từ $a \rightarrow v$.

Thuật toán Dijkstra

Khởi tạo: $k = 0$, $S_0 = \emptyset$, $L(a) = 0$, $L[v] = \infty$, $\forall v \neq a$.

Vòng lặp thứ k

1. Tìm đỉnh u không thuộc S_k có nhãn nhỏ nhất.
2. Bổ sung đỉnh u vào S_k , tức là $S_{k+1} = S_k \cup \{u\}$.
3. Điều chỉnh lại nhãn của các đỉnh v không thuộc S_{k+1} như sau:

Nếu $(u, v) \in E$ và $L(v) > L(u) + w(u, v)$ thì

$$L(v) := L(u) + w(u, v) \text{ và } \text{Truoc}[v] = u.$$

Thuật toán dừng khi $z \in S_k$, khi đó đường đi ngắn nhất từ a đến z là:

$$a \rightarrow a_1 \rightarrow a_2 \rightarrow \cdots \rightarrow a_p = z,$$

trong đó $z = a_p$, $\text{Truoc}[a_j] = a_{j-1}$, $\forall j = 1, \dots, p$.

Nhận xét: Khi $L(u)$ hữu hạn thì có đường đi từ a đến u có độ dài bằng $L(u)$ (bằng cách dùng hàm Truoc).

Chứng minh Thuật toán Dijkstra

Phương pháp: Chứng minh bằng quy nạp toán học theo k rằng điều kiện Dijkstra thỏa mãn với mọi $k = 1, \dots, n$. Gọi $L_k(v)$ là nhãn của đỉnh v ở sau bước lặp thứ k , theo cách dẫn nhãn của thuật toán, để thấy rằng $L_{k+1}(v) \leq L_k(v)$ với mọi $v \in V$ và mọi $k \geq 1$.

1. Bước cơ sở: $k = 1$, ta có $S_1 = \{a\}$, nhãn $L_1(u) = w(a, u)$ nếu u kề a , $L_1(u) = \infty$ nếu u không kề a , để thấy điều kiện (1),(2) thỏa mãn.
2. Bước quy nạp: Giả sử điều kiện Dijkstra đúng với k , do $S_{k+1} = S_k \cup \{u\}$ nên ta chỉ cần chứng minh (1) đúng với đỉnh mới u . Gọi $s = (a, a_1, \dots, a_m, \dots, a_p, u)$ là đường đi bất kỳ từ a đến u và a_m là đỉnh đầu tiên không thuộc S_k , tức là $a, a_1, \dots, a_{m-1} \in S_k$, suy ra $t = (a, a_1, \dots, a_{m-1}, a_m)$ là đường đi từ S_k đến a_m . Hiển nhiên $w(s) \geq w(t)$, theo điều kiện (2) ta có $w(t) \geq L_k(a_m)$ mà $L(a_m) \geq L(u)$ do u và a_m đều không thuộc S_k nhưng u có nhãn nhỏ nhất. Vậy $w(s) \geq L_k(u) \geq L_{k+1}(u)$, mặt khác, do $L_{k+1}(u)$ hữu hạn nên có đường đi từ a đến u có độ dài $L_{k+1}(u)$, vậy (1) đúng với $k + 1$.

Chứng minh Thuật toán Dijkstra(tiếp)

2. Bước quy nạp: Bây giờ ta chứng minh điều kiện (2) đúng với mọi đỉnh $v \notin S_{k+1}$. Thật vậy, gọi $s = (a, a_1, \dots, a_m, v)$ là đường đi bất kỳ từ S_{k+1} đến v , dễ thấy $a_m \in S_{k+1} = S_k \cup \{u\}$.
- Nếu $a_m \in S_k$ thì s là đường đi từ S_k đến v , theo giả thiết quy nạp, ta có $w(s) \geq L_k(v)$ mà $L_k(v) \geq L_{k+1}(v)$ suy ra $w(s) \geq L_{k+1}(v)$, vậy (2) đúng.
 - Nếu $a_m \notin S_k$ suy ra $a_m = u$, đặt $t = (a, a_1, \dots, a_{m-1}, u)$ ta có $w(s) = w(t) + w(u, v)$ mà theo (1) thì $w(t) \geq L_{k+1}(u)$ vậy:

$$w(s) \geq L_{k+1}(u) + w(u, v) \quad (*)$$

Mặt khác, theo cách dán nhãn đỉnh v ở bước thứ $k + 1$, ta luôn có

$$L_{k+1}(u) + w(u, v) \geq L_{k+1}(v) \quad (**)$$

(vì nếu ngược lại thì $L_{k+1}(v)$ đã được gán bằng $L_{k+1}(u) + w(u, v)$).

Vậy từ (*) và (**), ta được $w(s) \geq L_{k+1}(v)$, tức là (2) đúng.

Thuật toán Dijkstra-Giả mã

Proceduce (Đồ thị G có trọng số, một đỉnh a)

For $v \in V$

$$L[v] = \infty, \quad Truoc[v] = -1, \quad L[a] = 0, \quad S = \emptyset$$

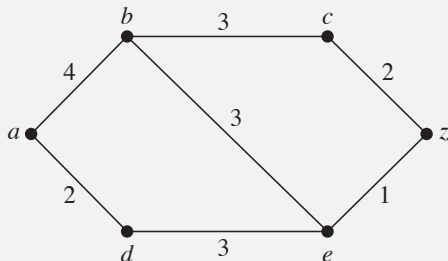
While $S \neq V$ do

1. Tìm đỉnh $u \notin S$ có nhãn $L[u]$ nhỏ nhất.
2. $S = S \cup \{u\}$.
3. For $v \in V \setminus S$ do

If $L[u] + w(u, v) < L[v]$ then

$$L[v] = L[u] + w(u, v), \quad Truoc[v] = u$$

Ví dụ



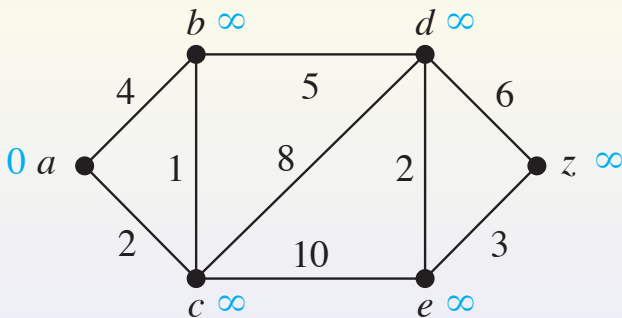
a	b	c	d	e	z
0	∞	∞	∞	∞	∞
	4 a	∞	2 a	∞	∞
	4 a	∞		5 d	∞
		7 b		5 d	∞
		7 b			6 e

- Đường đi ngắn nhất: $a \rightarrow d \rightarrow e \rightarrow z$ vì:

$$\text{Truoc}[z] = e, \text{Truoc}[e] = d, \text{Truoc}[d] = a.$$

Chú ý: Ở bước thứ k (ứng với dòng thứ k): số ứng với đỉnh (cột) nào là nhãn của đỉnh đó, chữ phía sau (nếu có) là đỉnh trước của nó trong đường đi ngắn nhất, ô vuông đánh dấu đỉnh được đưa vào tập S .

Ví dụ 1



Ví dụ 2

